Database Lab : Connection Pool Timeout Tuning & Governance Policy

Duration : 60 Min.

Scenario

As a Senior Platform Reliability Engineer at NIIT, application services frequently exhaust database connection slots due to unclosed sessions. You must inspect live session settings in PostgreSQL, configure session timeout parameters to prune zombie sessions, and establish a connection pool governance audit table.

Input Details

The environment has been pre-configured with the following local database platform instances:

- **Database Host:** `localhost` (Ubuntu Linux VM)
- **PostgreSQL Service:** `localhost:5432` | DB: `postgres` | User: `postgres` | Password: `PostgresPassword123!` | CLI: `psql`

Username & Naming Conventions

Your candidate prefix is: **`<prefix>`** (e.g. `student-exam123`).

Resource naming: - **Pool Governance Table:** `<prefix>_pool_governance`

Task Objectives

Perform the following configuration steps directly via `psql`:

1. Interrogate Live Database Session Metrics

- In `psql`, query `SHOW max_connections;` and inspect `SELECT count(*) FROM pg_stat_activity;`.

2. Create Connection Pool Governance Table

- Create table `<prefix>_pool_governance` with columns:
- `policy_id SERIAL PRIMARY KEY`
- `pool_name VARCHAR(50) NOT NULL`
- `recommended_max_pool_size INT NOT NULL`
- `connection_timeout_ms INT NOT NULL`
- `leak_detection_threshold_ms INT NOT NULL`
- `configured_by VARCHAR(50) NOT NULL`

3. Populate Sizing Parameters & Enforce Session Timeouts

- Insert a governance rule for `HikariPool-Main` with `recommended_max_pool_size = 25`, `connection_timeout_ms = 30000`, `leak_detection_threshold_ms = 60000`, and `configured_by = '<prefix>'`.
- Run `ALTER SYSTEM SET idle_in_transaction_session_timeout = '60s';` to protect backend sessions from starvation.

Verification

Once completed, run the verification script:

```
python run.py
```

The driver inspects `pg_settings` and queries `<prefix>_pool_governance` on `localhost:5432`.

Grading Criteria

Your performance will be evaluated based on the following test cases:

Test Case	Requirement	Marks
TC1	PostgreSQL Session & Capacity Metrics Interrogation	5 Marks
TC2	Pool Governance Table (_pool_governance) Creation	5 Marks
TC3	Pool Sizing & Leak Detection Threshold Parameters Population	5 Marks
TC4	Server-Side Idle Session Timeout Configuration Verification	5 Marks

Total Score: 20 Marks

Important Notes

- Replace `<prefix>` with your candidate prefix.
- All SQL commands must be committed in the live PostgreSQL instance.